AWS AI Image Annotation Platform
Project Report

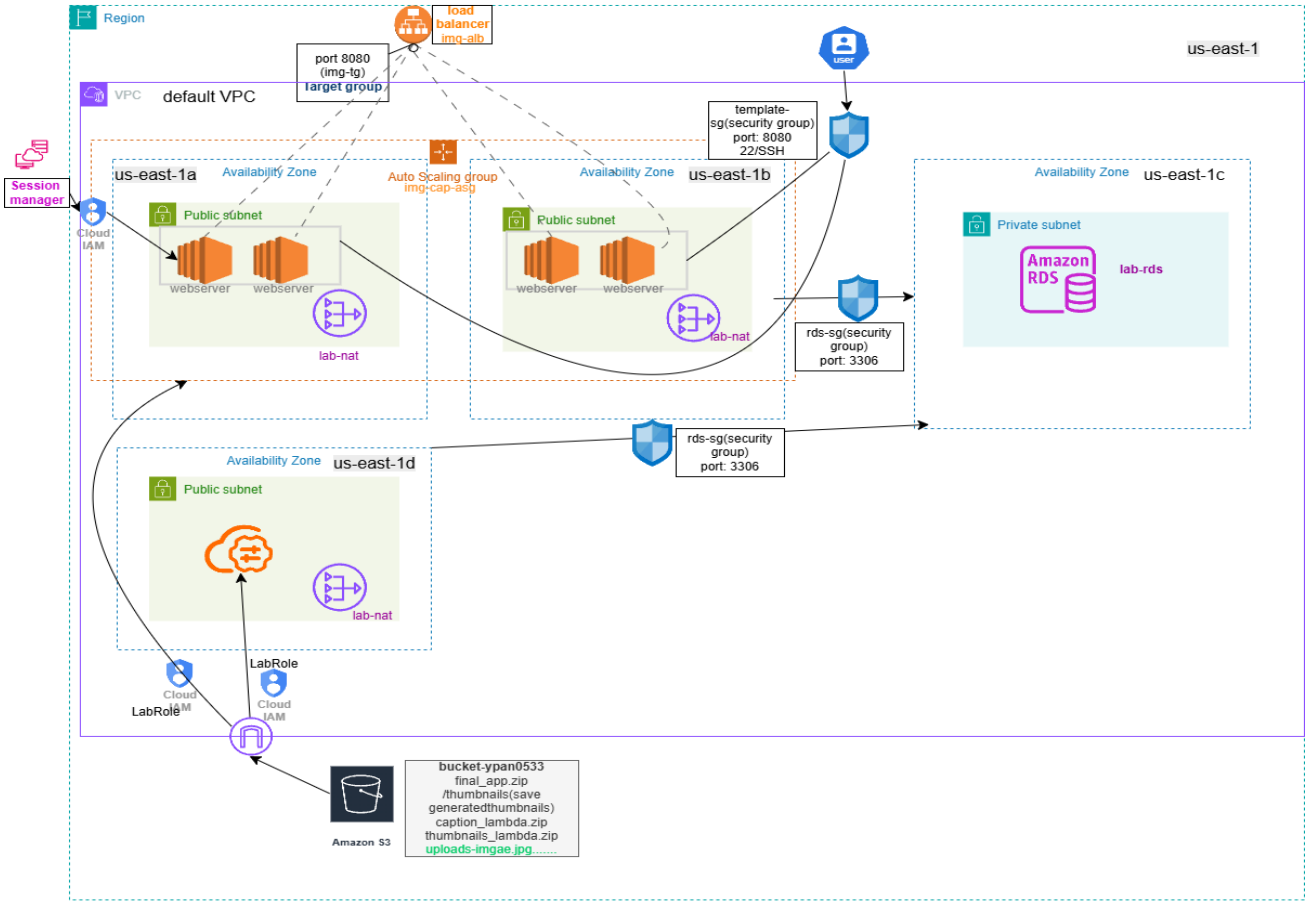
1. Introduction

This project aims to explain the deployment of a scalable and automated image processing platform on AWS cloudy, which enables users to upload images through a web interface. When an image is uploaded by users, it is stored in Amazon S3, which serves as the central data source for both metadata processing and thumbnail generation in gallery, while allows people to click the pictures' URLs to obtain the thumbnail original images. The upload event triggers two parallel backend processes: a **Lambda function** that uses the Gemini API to generate a caption and store it in an RDS MySQL database, and another **Lambda function** that generates a thumbnail image and saves it back to S3.

To manage user interaction and application delivery, key components will be combined into deployment for our deliverables: a Flask-based web application is hosted on **EC2** instances governed by an Auto Scaling Group (**ASG**). An Application Load Balancer (**ALB**) distributes incoming traffic among the instances and ensures fault tolerance via health checks. All components reside within our default **VPC**, which isolates resources across public and private subnets. Security is maintained through **existing IAM roles** (LabRole) and **security groups** that regulate access among services. Event-driven setting is achieved using **EventBridge** to route **S3** events to the appropriate **Lambda functions**, establishing a well-integrated and modular system architecture.

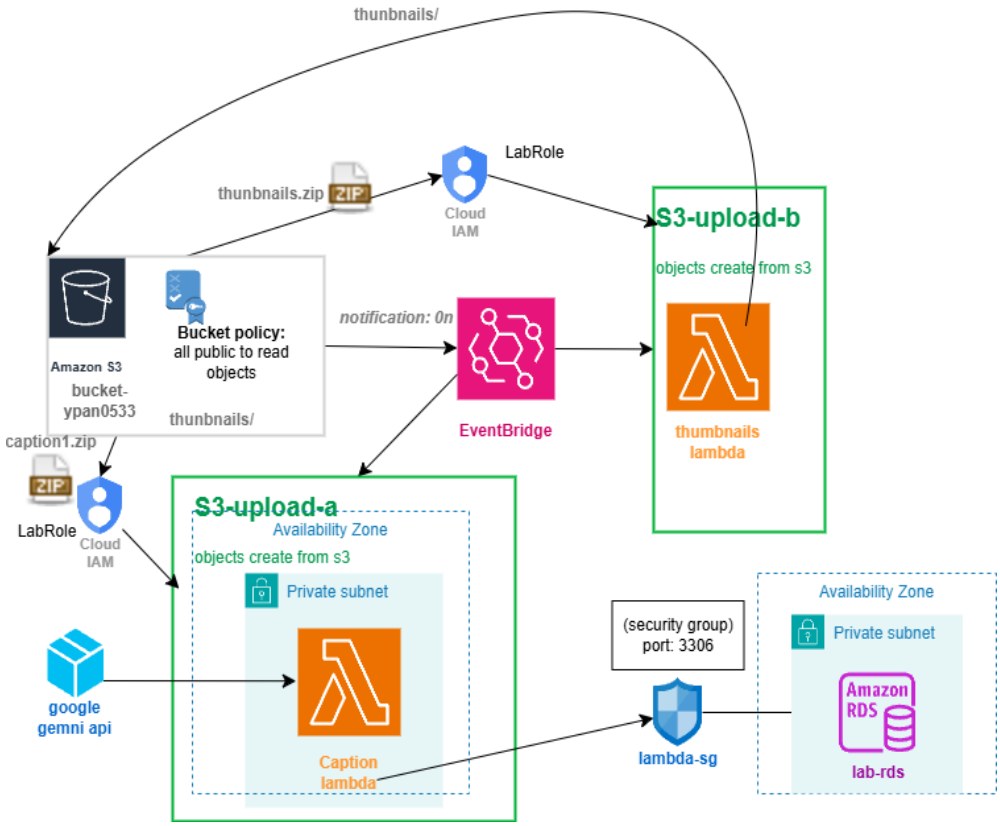
2. Architecture Diagram

2.1. Web application architecture



The web application is deployed in the us-east-1 region using a scalable and secure architecture. An Application Load Balancer (ALB) routes traffic to EC2 instances managed by an Auto Scaling Group (ASG) across multiple public subnets. Uploaded images are stored in Amazon S3, while metadata is saved in an RDS MySQL database within a private subnet. Security is enforced through VPC subnets, security groups, and IAM roles, with a Bastion Host accessed via Session Manager enabling secure administration without exposing SSH.

2.2. Serverless architecture



The serverless architecture is designed around Amazon S3 as the event trigger, with AWS Lambda functions handling backend processing in a fully managed, scalable environment. When an image is uploaded to the S3 bucket (bucket-ypan0533), an event is published to Amazon **EventBridge**, which routes it to two separate **Lambda functions**. The caption Lambda, deployed in a private subnet, generates image captions using the **Google Gemini API** and stores metadata in an RDS MySQL database. The thumbnail Lambda generates resized versions of the uploaded images and saves them to the thumbnails/ folder in the same bucket. **IAM** roles (LabRole) provide permissioned access for each component, while EventBridge decouples services for better modularity. The entire workflow avoids server provisioning and leverages VPC security groups and subnets for network isolation.

3. Web Application Deployment

3.1. Compute Environment

The computing environment is deployed using an **Auto Scaling Group (ASG),** img-cap-asg**,** to manage EC2 instances dynamically based on traffic load. These EC2 instances run a Flask-based web application that supports image uploads and displays annotated results. The ASG is configured with a minimum of 1 and a maximum of 2 instances, and it spans two public subnets located in Availability Zones *us-east-1a* and *us-east-1b*, ensuring high availability and load distribution across zones.

To automate deployment, each EC2 instance is launched using a **Launch Template** that references the latest application version. During boot, a **User Data** script set on template, is executed to download a zipped

application package from an **S3 bucket**, unzip it, install required Python dependencies, and start the Flask server automatically. This approach allows rapid updates by simply uploading a new zip package and updating the Launch Template version, without manually baking new **AMIs**.

All instances are deployed inside the **default VPC** in the *us-east-1* region, and public IP addresses are automatically assigned. These instances reside in public subnets associated with an **Internet Gateway**, allowing them to access **external services** such as the Gemini API and MySQL.

Security is enforced using **Security Groups** and **IAM Roles**. The EC2 instances are associated with a security group (template-sg) that allows **inbound HTTP traffic** on port 8080 for the web application. Port 22 is not open, as administrative access is securely handled through SSH AWS Systems Manager Session Manager, eliminating the need for SSH and key pair management. The attached IAM role (LabRole) grants permission for the EC2 instances to interact with AWS services such as S3 and RDS securely, without hardcoded credentials.

Administrative operations, including initial environment setup and application uploads, are conducted via **AWS Cloud9**, which was used to prepare and transfer application files to **S3**. The Cloud9 environment is deployed in a random AZ within us-east-1 and provides secure access to EC2 instances using **Session Manager.**

Traffic to the application is routed through an **Application Load Balancer (ALB**) that listens on port 8080 and is deployed across the same two public subnets as the ASG. The ALB is associated with a **Target Group** (img-cap-tg) which dynamically registers EC2 instances launched by the ASG. The Target Group uses the instance protocol and port 8080, ensuring direct routing to each web server instance. This configuration allows seamless scaling and instance replacement without manual intervention.

Health checks are configured within the Target Group to continuously monitor the status of EC2 instances. The health check uses the **HTTP protocol** and probes the root path (/) at regular intervals. If an instance fails to respond with a 200 OK status within the expected timeout and threshold, it is marked as **unhealthy** and automatically removed from the load balancer's active pool. Once it recovers and passes health checks consistently, it is re-registered. This mechanism guarantees that only healthy instances serve user traffic, maintaining application reliability and minimizing downtime.

Together, the ALB and Target Group ensure fault-tolerance, zone-aware load distribution, balancing incoming requests across multiple Availability Zones and automatically excluding unhealthy targets from rotation.

3.2. Database Environment

The application uses an **Amazon RDS (**lab-rds**)** for MySQL instance to store metadata such as uploaded image filenames, generated captions, and timestamps. The RDS database is provisioned within a private subnet in Availability Zone *us-east-1c*, ensuring that it is not publicly accessible from the internet. This subnet is part of the same **default VPC** as the rest of the application, enabling secure internal communication via private IP.

- **Configuration:** In terms of **configuration**, the **RDS** instance uses default MySQL engine settings with a single DB instance (no multi-AZ) and general-purpose SSD (gp2) storage for cost consideration and training balance The database is initialized with a single table to store image captioning metadata, and the schema setup is performed using a SQL script executed via our created Cloud9 environment or included in the **EC2 user data.**
- **Access Policy:** Access to the RDS instance is tightly controlled using a dedicated **security group(**rds-sg**)**, which permits inbound traffic only on port 3306 from trusted internal sources (~cloud 9 env IP and lambda function). These include EC2 instances within the Auto Scaling Group and the caption Lambda function, both of which reside within the VPC. No external IP ranges are allowed, enforcing strict network-level isolation. Database credentials such as host address, username, and password are injected into the EC2 and Lambda environments using **environment variables**, avoiding hardcoded secrets.
- **Integration with Application:** The application connects to the database using the mysql-connector-python library on EC2 and the appropriate Python MySQL libraries within Lambda. Both components query and insert metadata directly via standard SQL operations. Monitoring of the RDS instance is

handled via Amazon CloudWatch, allowing visibility into performance metrics such as CPU utilization, free storage space, and connection count.

Overall, the RDS instance is securely integrated into the application architecture, providing a reliable relational backend with well-defined access boundaries and no public exposure.

3.3. Storage Environment

The application uses Amazon S3 as a multi-purpose storage solution through the bucket **bucket-ypan0533**, which supports image uploads, thumbnail outputs, EC2 initialization, and Lambda function deployment. The bucket plays a central role in both the web application architecture and the serverless processing pipeline.

1. **Image Uploads and Outputs**
   User-uploaded images are sent from the web application directly to the root of the S3 bucket. Once uploaded, turn on **EventBridge notification**, it will capture the ObjectCreated event and triggers two Lambda functions:
   - **The caption Lambda** generates a descriptive caption using the Gemini API and stores the metadata in RDS.
   - **he thumbnail Lambda** creates a resized image and stores it under the dedicated thumbnails/ folder (e.g., *thumbnails/image1_thumb.jpg*), separating processed results from raw uploads.
2. **EC2 Application Deployment**
   The bucket also contains a compressed application package named final_app.zip, which includes the web application source code and dependency files. This zip file is downloaded automatically during EC2 instance's initialization via a User Data script, enabling consistent deployment without pre-built AMIs.
3. **Lambda Deployment Packages**
   To support serverless processing, the bucket hosts two zipped Lambda deployment packages:
   - **caption1.zip:** contains the handler code and required Python packages for the caption Lambda function.
   - **thumbnails.zip:** includes the thumbnail generation handler for thumbnails lambda function and its dependencies.
   - These deployment packages were uploaded manually and linked during Lambda creation, allowing for quick updates and modular development.
4. **Security and Access**
   The bucket is protected by a custom S3 Bucket Policy that grants access only to specific IAM roles used by EC2 and Lambda components. Public access is fully blocked. File access and event handling are controlled through fine-grained permissions, ensuring only trusted AWS services can interact with stored assets. how the Lambda function is packaged (e.g. as a simple script, a zip file or a container image) and deployed (e.g. through management console, cli or CloudFormation) and the settings to allow it interact with S3, RDS and Gemini API.

4. Serverless Component Deployment

4.1. Event-Driven Architecture

The application leverages an **event-driven architecture** to automate image processing workflows immediately after file uploads, eliminating the need for polling or manual intervention. This is achieved using **Amazon S3 event notifications** integrated with **Amazon EventBridge**, which serves as the event routing mechanism for invoking backend Lambda functions.

When a user uploads an image to the S3 bucket (**bucket-ypan0533**), an ObjectCreated event is emitted by turning on the eventbridge notification, which event is captured by **EventBridge**, it is configured to match file creation events and route them to two distinct **Lambda functions**. Meanwhile, all public to read object permission will be applied by creating a **bucket policy** in S3.

- The **caption Lambda** is triggered upon image upload. It downloads the image from S3, sends it to the Gemini API for caption generation, and writes the resulting metadata (caption, filename, timestamp) to the

RDS MySQL database. The function runs within a private subnet and accesses the database through a security group with port 3306 enabled.

- The **thumbnail Lambda** is also triggered by the same event and generates a resized version of the uploaded image. The thumbnail is then saved back to the same bucket under the thumbnails/ folder, following a clear folder structure for separation of original and processed files.

Both functions are deployed using ZIP packages (caption1.zip and thumbnails.zip) stored in the S3 bucket. These packages include the Python handler code and all necessary dependencies. Lambda functions are granted access to required AWS services through an **IAM** role (**LabRole**) that enables interaction with S3 and RDS.

This architecture decouples image upload from processing, allowing both functions to execute independently, in parallel, and with minimal latency. **EventBridge** ensures flexible routing and easy implementation of equivalent trigger function.

4.2. Annotation Function

The annotation function responsible for generating image captions is deployed as a **Lambda function,** using a ZIP package named caption1.zip, which is manually uploaded to the S3 bucket (**bucket-ypan0533**). The ZIP file includes the handler script lambda_caption.py and all required **Python dependencies**, such as boto3, pillow, google-generative Ai, and pymysql, bundled locally and zipped using *pip install -t .* in a **Cloud9 environment.**

The Lambda function is created through the **AWS Management Console**, with the runtime set to **Python 3.9**, and the handler configured as **lambda_caption.lambda_handler.** The function runs inside the **default VPC**, allowing secure access to the **RDS database.** Once a user uploads an image to the S3 bucket, an **ObjectCreated** event is captured by **Amazon EventBridge** and forwarded via the rule **s3-upload-a** to the annotation Lambda function. This event-driven setup allows the function to process new images automatically.

- **Functionality:**
  Once triggered, the caption Lambda function downloads the newly uploaded image from the S3 bucket and uses the google-generative ai Python SDK to send the image to the Gemini API. The API responds with a generated textual description of the image content. The Lambda function then parses this response, extracts the caption, and saves both the caption and associated metadata, such as image filename, timestamp, and caption confidence (if applicable), into the RDS MySQL database.
  This caption is not only stored for backend logging and traceability but is also intended to be rendered alongside the uploaded image in the front-end web interface. Users accessing the gallery can view each image along with its AI-generated caption, providing enhanced context and interactivity. The function thereby serves as a key integration point between AI inference, cloud storage, and user-facing data display.

To enable **secure and functional integration** with **cloud resources**:

- The function assumes the IAM role LabRole, which grants it permission to access **S3** (read uploaded images), write logs to **CloudWatch**, and connect to **RDS** through the VPC using a security group that allows port 3306.
- The **Gemini API key** is passed to the function via environment variables (GEMINI_API_KEY) and securely retrieved in the handler code at runtime using os.environ.
- Set Caption Lambda in AZ *us-east-1e* and *us-east-1f* Private subenet and using **lambda-sg** security group to access the resource.

Upon invocation, the function downloads the image from S3, sends it to the **Gemini API** to generate a caption, and stores the resulting text along with metadata (e.g., filename, timestamp) into the RDS MySQL database.

4.3. Thumbnail Generator

The thumbnail generation function is deployed as an **AWS Lambda function** using a ZIP archive named **thumbnails.zip**, which is manually uploaded to the S3 bucket (**bucket-ypan0533).** This ZIP package contains the **handler script** lambda_thumbnails.py and all required dependencies, including pillow and boto3. The package was built inside a **Cloud9 environment** using pip install -t . followed by compression into a deployment-ready ZIP file.

The function is created through the **AWS Management Console**, using **Python 3.9** as the runtime. The handler is configured as lambda_thumbnails.lambda_handler, and the function runs within the **default VPC**. However, it does not require RDS or external APIs; it only processes files within Amazon S3.

- **Functionality:**
  When a user uploads an image to the S3 bucket, an ObjectCreated event is captured by **Amazon EventBridge** and routed through the rule **s3-upload-b**, which triggers this Lambda function. Once invoked, the function retrieves the original image from the bucket, uses the **Pillow** library to generate a resized version (thumbnail), and saves the processed file to the thumbnails/ directory in the same bucket. The output filename follows a naming convention (e.g., thumbnails/image1_thumb.jpg) to ensure clarity and prevent overwrite conflicts.
  Additionally, since the bucket has a public read policy enabled for the thumbnails/ path, the thumbnail can be accessed directly via a **public S3 URL**, which allows the image to be displayed on the front-end web interface. By double-clicking on a processed image or following its generated link, users can instantly preview the result. This function thus supports real-time user interaction by integrating backend image processing with front-end accessibility.

To enable this functionality:

- The function is granted permission to read from and write to S3 through the IAM role **LabRole**. Specifically, this role allows actions such as s3:GetObject, s3:PutObject, and s3:ListBucket on the **bucket-ypan0533**.

- The function does not rely on any external credentials or APIs, making it lightweight and fast to execute within isolated compute resources.

This Lambda function supports the application's serverless processing pipeline by automatically handling image resizing without requiring any EC2 or manual intervention. Its **use of EventBridge for triggering** and S3 for both input and output provide a scalable, decoupled design.

5. Auto Scaling Test Observation

To verify the responsiveness of our Auto Scaling Group (ASG) under load, we conducted a performance test using ApacheBench (ab) from a provisioned EC2 instance. This instance was accessed securely through EC2 Session Manager (Connect) inside Cloud9, eliminating the need for SSH keys or public IPs. The stress testing tool ab was installed in the session terminal.

The following command was executed:

```
ab -n 1000 -c 100 http://<ALB-DNS>:8080/
```

This simulated a workload of 1,000 HTTP requests with 100 concurrent connections targeting the Application Load Balancer (ALB) endpoint on port 8080. The goal was to trigger scaling actions and observe system responsiveness.

- **Scaling Policy and Behavior:** The Auto Scaling Group (img-cap-asg) used a Target Tracking Scaling policy with the following configuration:

Policy Type	Metric	Target Value	Warm-up Period	Scale-in
Target Tracking Scaling	Average CPU utilization	30%	300 seconds	Enabled

Explanation: The table describes the configuration of the Auto Scaling Group's target tracking policy. It is set to maintain an average CPU utilization of 30%. When the CPU usage exceeds this threshold, the group will automatically add a new EC2 instance. The newly launched instance is excluded from the CPU metric calculation for the first 300 seconds (warm-up period). Scale-in is enabled, allowing the group to remove instances when load decreases.

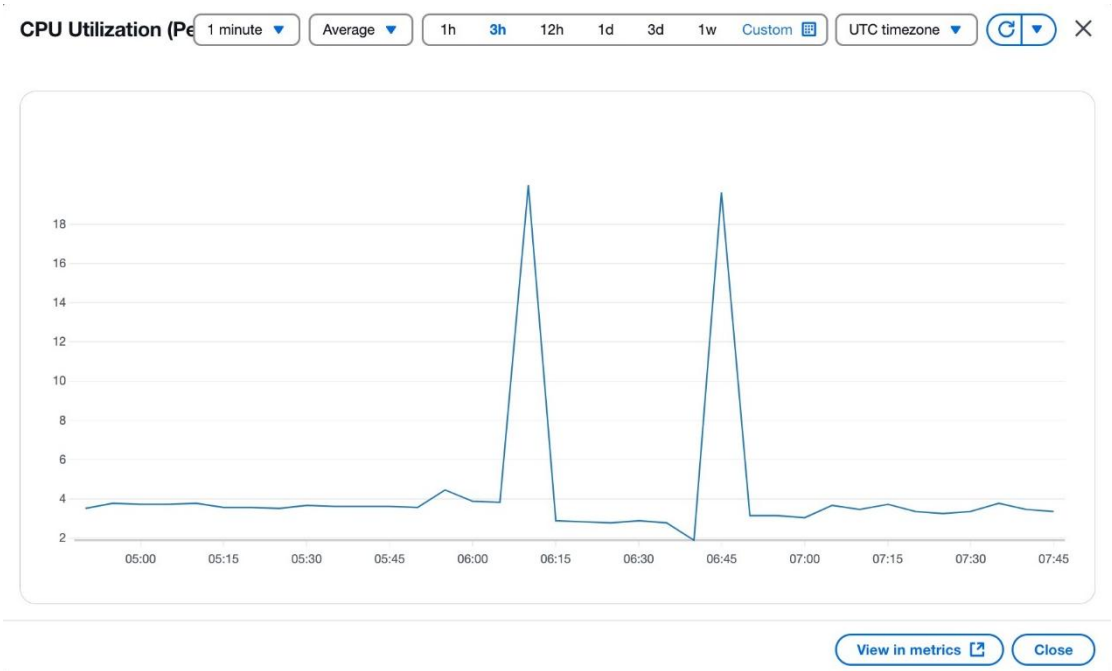
- **ALB health Check Configuration:** The ALB health check configuration is summarized below:

Parameter	Protocol	Port	Path	Healthy threshold	Unhealthy Threshold	Interval	Timeout	Success Code
Value	HTTP	8080	/	5	2	30 seconds	5 seconds	200

Explanation: The table outlines the health check settings applied to the Application Load Balancer (ALB). It checks the root path (/) of each EC2 instance every 30 seconds using HTTP on port 8080. An instance is considered healthy after five consecutive successful checks (returning HTTP 200), and marked unhealthy after two consecutive failures. This configuration ensures that only responsive instances serve traffic.

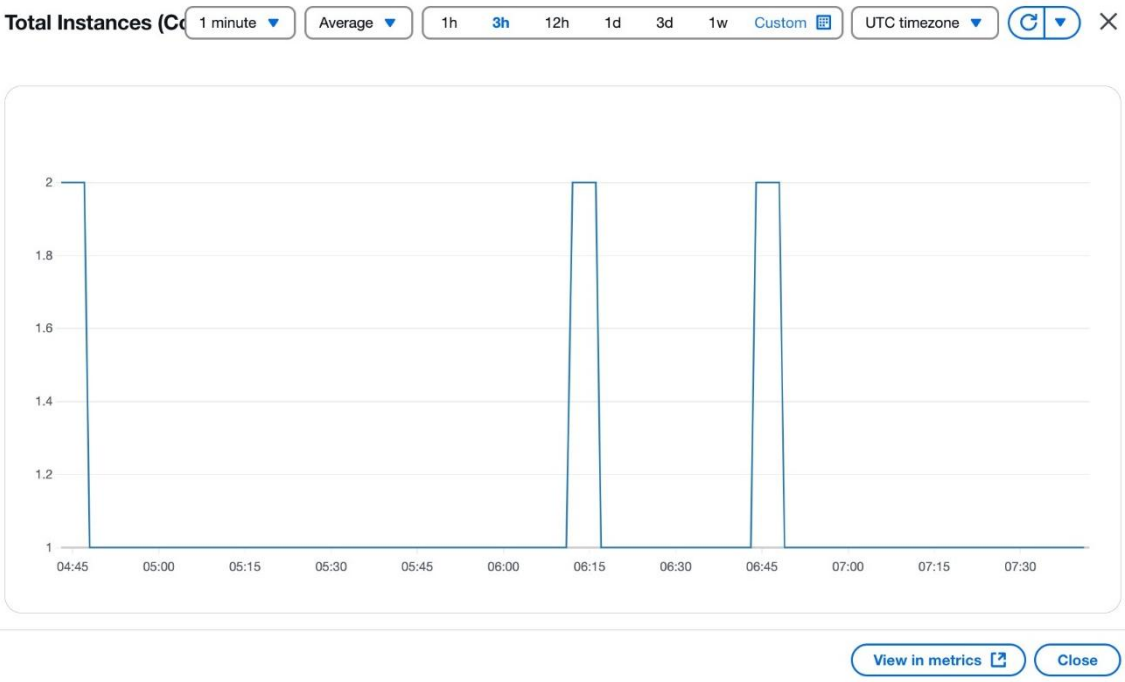
Visualization of Monitoring and Analysis:

1. **Average CPU Utilization**



- **Description:**
  The line chart displays the average CPU utilization of EC2 instances over a 3-hour period, measured in 1-minute intervals. Two significant spikes are observed—one around 06:10 UTC and another around 06:45 UTC—both reaching just above 18%.
- **Analysis:**
  These two spikes correspond to the simulated workload applied using ApacheBench. During both events, the CPU utilization exceeded the ASG target threshold of 30%, triggering **scale-out.** The temporary rise and fall of CPU usage validate the system's ability to respond to demand surges. The sharp drops following each spike suggest that the new EC2 instances successfully began serving traffic, redistributing load across both instances. This behaviour confirms that the system's scaling logic worked as intended, dynamically responding to increased workload. The cooldown periods also highlight how the ASG avoids overreacting to short-term spikes, balancing responsiveness and resource efficiency.

2. **Total Instance Number**



- **Description:**
  This chart illustrates the number of EC2 instances managed by the Auto Scaling Group (ASG) overtime. It shows the instance count increasing from 1 to 2 during two distinct periods and then returning to 1.
- **Analysis:**
  The instance count chart precisely mirrors the CPU activity timeline. During each spike, the Auto Scaling Group initiated **scale-out**, increasing the number of EC2 instances from 1 to 2. This demonstrates that the scaling trigger logic (target tracking policy with 30% CPU threshold) was properly configured and functional.
  Once the load subsided, the instance count returned to 1, showing that scale-in was also successful. The delay between the spike and **scale-in** confirms that the system respected the warm-up period and stability thresholds before terminating the additional instance. Together with the CPU chart, this graph confirms that the ASG dynamically maintained computing resources in accordance with real-time demand.

6. Conclusion

The project successfully implemented a modular and scalable image annotation system on AWS, merging a Flask-based EC2 environment with an Auto Scaling Group and serverless Lambda functions orchestrated via EventBridge to achieve robustness and maintainability.

During deployment, we encountered several **significant challenges**:

- **Broken Thumbnail Display:** During testing, the web application occasionally displayed broken image icons instead of thumbnails. Investigation revealed several root causes: thumbnails were being uploaded without proper Content-Type headers, the object keys did not match the expected paths used by the frontend, and S3 bucket permissions prevented public access. These issues were resolved by explicitly setting the correct MIME type (image/jpeg) when saving thumbnails, ensuring the object keys aligned with the HTML image tags, and updating the S3 bucket policy to allow public read access for files in the thumbnails/ prefix.

- **S3 Access Permissions:** Early testing revealed 403 Forbidden errors when Lambda attempted to show the thumbnails on our website from the S3 bucket. By creating our S3 bucket policy—granting read/write access specifically to the public and verifying IAM trust relationships, we restored proper access without over-permission.
- **VPC Connectivity Issues:** The caption Lambda, running inside private subnets, couldn't reach the Gemini API endpoint directly. We corrected this by attaching a NAT Gateway to the public subnet and updating route tables for the private subnets (us-east-1e/us-east-1f), allowing outbound internet access while keeping the function isolated.
- **Auto Scaling Behavior Tuning:** During load testing, we observed delayed scale-out responses with ApacheBench. Adjusting our Target Tracking policy to maintain a 30% average CPU utilization and reducing evaluation intervals improved responsiveness, enabling the ASG to add or remove EC2 instances more promptly.
- **EventBridge Rule Precision:** We initially used broad event patterns that triggered both lambdas on every upload. Refining our EventBridge rules—matching specific S3 key prefixes (uploads/ for captions and uploads/ for thumbnails triggers)—allowed us to route events appropriately without unnecessary invocations.

Throughout these iterations, we conducted systematic debugging—using CloudWatch Logs, Lambda test events, S3 bucket policy created and manual AWS CLI checks—to trace errors and validate solutions. Each fix reinforced both security and cost-efficiency, shaping a production-aligned architecture.

In Conclusion, this deployment underscores the value of precise event-driven design, scoped IAM permissions, and responsive auto scaling policies. Through the seamless integration of EC2, Lambda, S3, RDS, and EventBridge, we established a secure, scalable, and cost-effective image annotation platform. By adhering to AWS best practices, the system achieves high availability and real-time responsiveness, offering a robust foundation for future enhancements and production-scale deployment. While the current architecture performs reliably under simulated load and supports core functionality, several enhancements could improve system robustness and extensibility. These include integrating authentication and access control for image uploads, enabling multi-language support for captions, and implementing versioning in S3 to track image edits or updates. Additionally, monitoring can be expanded with CloudWatch dashboards and alerts for more granular visibility. Moving forward, containerizing the application or migrating to AWS Lambda-based frontends could further reduce operational overhead and improve scalability.